

A Python Framework for Solving 1-Dimensional Nozzle Flows using Quantum Linear Equation Solvers

Leigh Lapworth
Rolls-Royce

July 11, 2025

Abstract

This article describes a hybrid quantum-classical testing framework for the 1-dimensional flow through a convergent-divergent nozzle. The CFD solver models both low-speed incompressible and transonic compressible flow. It also offers both the segregated SIMPLE solver and an implicit coupled solver for both flow types. The core solver is written in C and is accessible via a Python wrapper using the *ctypes* interface. This allows the solver to be tested with a range of quantum computing APIs. The solver is controlled using a simple XML file. The Python interface using a simple get and put interface to retrieve the CFD matrix and right hand side vector from the CFD discretisation and return the QLES solution to update the flow field. There are a range of solver options that can be used to generate matrices with up to 8 diagonals using the coupled compressible solver. The standard pressure correction solver has 3 diagonals. The coupled incompressible solver has 6 diagonals and gives rapid convergence which is useful when emulating the quantum circuit is time-consuming.

Notation

γ	ratio of specific heats, $\gamma = \frac{c_p}{c_v}$
c_p	specific heat capacity at constant pressure
c_v	specific heat capacity at constant volume
M	Mach number = velocity/speed of sound, $M = \frac{u}{\sqrt{\gamma R t}}$
p	static pressure
P_0	stagnation pressure
R	perfect gas constant, $R = c_p - c_v$
t	static temperature
T_0	stagnation temperature
u	axial velocity

Contents

1	Introduction	3
2	Nozzle CFD	3
2.1	Boundary Conditions	4
2.1.1	Compressible inflow condition	4
2.1.2	Incompressible inflow condition	5
2.2	CFD discretisation	5
2.2.1	Incompressible coupled solver	6
2.2.2	Compressible coupled solver	7
2.2.3	Incompressible SIMPLE solver	9
2.2.4	Compressible SIMPLE solver	10
3	Core CFD solver	11
3.1	Building the C code	11
3.1.1	Linux based systems	11
3.1.2	Windows based systems	11
3.2	Running the code	11
3.3	Python-C interface	12
3.3.1	Data instantiation	12
3.3.2	Data exchange	12
3.4	Linking to a quantum linear equation solver	14
3.5	Input file	14
3.5.1	Overview	14
3.5.2	Mesh data	15
3.5.3	Incompressible flow, SIMPLE solver	15
3.5.4	Incompressible flow, coupled solver	15
3.5.5	Compressible flow, SIMPLE solver	16
3.5.6	Compressible flow, coupled solver	16
3.6	Output files	17
3.6.1	File naming convention	17
3.6.2	Convergence history	17
3.6.3	Flow field	18
4	Analysing the results	18
4.1	Convergence histories	18
4.2	Flow field	19
5	Summary	19

1 Introduction

The 1D nozzle solver provides a flexible Python interface for testing quantum linear equation solvers on fluid flow type problems. The core CFD solver, written in C, is linked via a library to enable developers to focus on their quantum solver.

The benefit of the nozzle system is that it allows researchers to investigate the feedback between the inner linear system, solved using a quantum algorithm, and the outer classical non-linear update. [5] has shown the performance of HHL [4], relative to a classical linear solver, must be assessed within the context of the convergence of the non-linear CFD solver. For HHL, there is a specific iterative feed-forward mechanism associated with the under-resolution of the eigenvalue spectrum. Similar behaviour is anticipated with other linear solvers such as Quantum Singular Value Transformation (QSVT) [6, 2, 1], albeit as a result of different mechanisms.

Although the solver is only 1 dimensional, there are a range of solver options that can be used to generate matrices with up to 8 diagonals using the coupled compressible solver. The standard pressure correction solver has 3 diagonals. The coupled incompressible solver has 6 diagonals and gives rapid convergence which is useful when emulating the quantum circuit is time-consuming.

The plotting routines enable easy comparison of classical and quantum results.

2 Nozzle CFD

Throughout this document the test case is referred to as a *nozzle*. This is the more common terminology for transonic flows. For incompressible flows, *channel* is more commonly used, but we use a single terminology for both flow regimes.

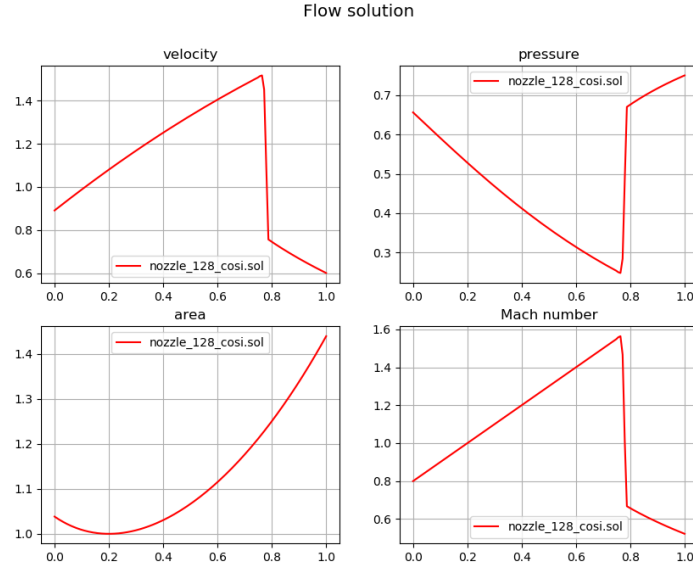


Figure 1: Solution for compressible nozzle with 128 stations.

Figure 1 shows the converged flow solution for a transonic compressible test case. The bottom left figure shows the area of the channel, A , which is derived from the isentropic flow relationships:

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma + 1}{2(\gamma - 1)}} \quad (1)$$

where M is the Mach number, γ is the ratio of specific heats and A^* is the area at the throat where $M = 1$. In this test case, all variables are non-dimensionalised so that $A^* = 1$. The length of the nozzle is always set to 1 and the user specifies an inlet Mach number, M_{in} . To set-up the nozzle area distribution, the outlet Mach number is set to $M_{out} = 1 + M_{in}$. The axial stations are set according an equal Mach number of increment of $\delta M = 1.0/(nx - 1)$ where nx is the number of axial stations in the mesh. The area

at each station is set from Equation (1). Note that the area equation is for isentropic, i.e. reversible, flow where the Mach number increases linearly. In the absence of a shock wave this gives the CFD solution. However, the interest is in the case when there is a shock wave within the nozzle as shown in the bottom right of Figure 1.

For incompressible flows, the channel area is an arc of a circle with a specified area contraction, typically 10% at the midpoint, relative to the inlet and exit areas which are both set to 1.0. The area distribution is not shown but follows the shape of the pressure distribution.

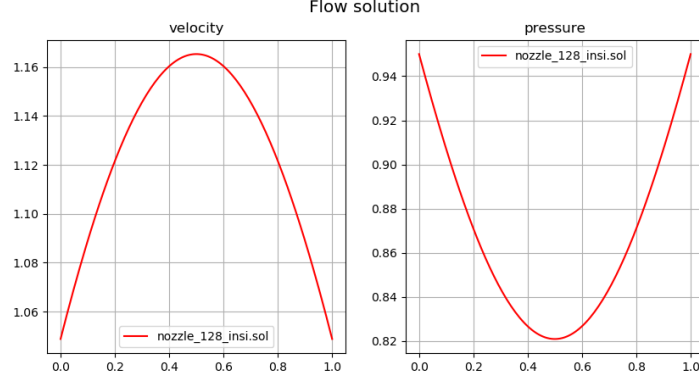


Figure 2: Solution for incompressible nozzle with 128 stations.

2.1 Boundary Conditions

In addition to setting the geometry of the nozzle, we also need to provide boundary conditions at the entrance and exit to the nozzle. These must be specified in a way that is consistent with that signals propagate within the flow.

Following Figure 3, a velocity boundary condition is set at the inlet as the velocity updates are largely convected in the flow direction. a pressure boundary condition is set at the outlet, as pressure signals propagate in all directions at the speed of sound, relative to the flow field. Setting an exit pressure boundary condition is only physically valid when the exit flow velocity is subsonic. This is case for both incompressible and compressible flows used here and, hence, both have the same form of outlet condition:

$$p|_{i=n} = p_{exit} \quad (2)$$

That is, p at the exit station, $i = n$ has a fixed value, p_{exit} . Although, incompressible and compressible flows have the same form of exit boundary condition, the values are different. The compressible solution in Figure 1 has $p_{exit} = 0.75$. Whereas, the incompressible solution in Figure 2 has $p_{exit} = 0.95$.

For both compressible and incompressible flows, the values of velocity at the exit and the pressure at the inlet are computed from the solution of the flow equations. At the inlet, the computed inlet value of pressure is used to set an inlet velocity according to the inflow boundary condition.

2.1.1 Compressible inflow condition

For compressible flows, the isentropic relationships give:

$$\left(\frac{T_0}{t}\right) = \left(\frac{P_0}{p}\right)^{\frac{\gamma-1}{\gamma}} \quad (3)$$

Where T_0 and P_0 are the stagnation temperature and pressure which are constant in isentropic, i.e. shock-free, flows. The flow upstream of the nozzle inlet is taken as isentropic and T_0 and P_0 are user-specified boundary values. For non-dimensionalised conditions, as used here, it is usual to set them both to unity.

Rearranging Equation (3) gives:

$$t = T_0 \left(\frac{p}{P_0}\right)^{\frac{\gamma-1}{\gamma}} \quad (4)$$

In addition, we have conservation of energy which, for isentropic flow, can be expressed in terms of temperature as:

$$T_0 = t + \frac{1}{2c_p}u^2 \quad (5)$$

And can be rearranged to give:

$$u = \sqrt{2c_p(T_0 - t)} \quad (6)$$

Equation (4) and Equation (6) allow the inlet velocity to be updated from the value of pressure and the specified inlet stagnation conditions.

2.1.2 Incompressible inflow condition

For incompressible flow, we cannot use the isentropic relations as strictly the Mach number is zero or, more accurately, the speed of sound is infinite. Instead, we use Bernoulli's equation:

$$P_0 = p + \frac{1}{2}\rho u^2 \quad (7)$$

From which, the inlet velocity can be set from the computed inlet static pressure and the specified inlet stagnation pressure:

$$u = \sqrt{\frac{2}{\rho}(P_0 - p)} \quad (8)$$

2.2 CFD discretisation

Since, the focus of this test case is not CFD development, we use the discretisation scheme of [3]. In this scheme, the velocities and pressures are stored at the same set of nodes with upwinded momentum control volumes and downwinded continuity control volumes. These are shown in Figure 3a and Figure 3b respectively.

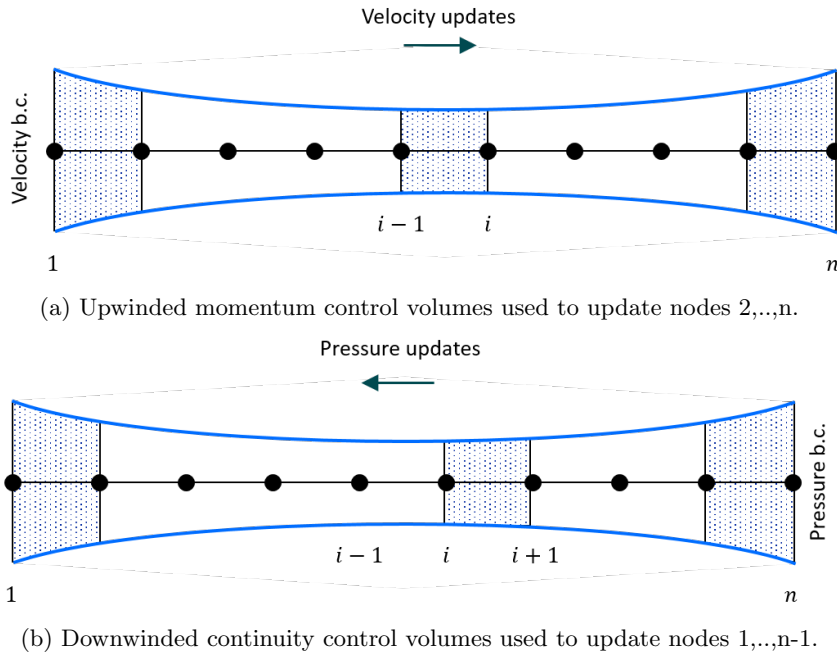


Figure 3: Control volume setting for momentum and continuity equations.

The 1-dimensional steady state equations to be solved are:

$$\begin{aligned} \rho u \frac{\partial u}{\partial x} &= -\frac{\partial p}{\partial x} \\ \frac{\partial \rho u}{\partial x} &= 0 \end{aligned} \quad (9)$$

For compressible flows, the following equations are also applied:

$$\begin{aligned} p &= R\rho t \\ T_0 &= t + \frac{1}{2c_p}u^2 \end{aligned} \quad (10)$$

where R is the gas constant, c_p is the specific heat at constant pressure, and T_0 is the constant total temperature which is set in 1.0 in non-dimensional units. This equation represents the conservation of energy.

The differential equations are discretised on the 1-dimensional mesh and updated iteratively via corrections:

$$\begin{aligned} |u\rangle &\leftarrow |u\rangle + |\delta u\rangle \\ |p\rangle &\leftarrow |p\rangle + |\delta p\rangle \\ |\rho\rangle &\leftarrow |\rho\rangle + |\delta \rho\rangle \end{aligned} \quad (11)$$

For incompressible cases, the density is fixed at a non-dimensional value of 1.0 and is not updated as part of the solution.

The coupled iterative scheme is the most conceptually straightforward and will be described first

2.2.1 Incompressible coupled solver

For incompressible flow, the discrete version of momentum equation is Equation (9) is:

$$\rho u_{i-\frac{1}{2}} \frac{u_i - u_{i-1}}{x_i - x_{i-1}} = -\frac{p_i - p_{i-1}}{x_i - x_{i-1}} \quad (12)$$

Note that we have followed [3] in using a finite difference discretisation for the momentum equation.

It is common practice in CFD to form an *inner* linearised system to solve for the corrections. The only non-linear term in Equation (12) is the term on the left hand side of the first equation. Following [3], this is linearised using a fixed point iterative scheme for the momentum equation to give:

$$\begin{aligned} \rho u_{i-\frac{1}{2}}(u_i + \delta u_i - u_{i-1} - \delta u_{i-1}) &= -(p_i + \delta p_i - p_{i-1} - \delta p_{i-1}) \\ \rho u_{i-\frac{1}{2}}(\delta u_i - \delta u_{i-1}) + (\delta p_i - \delta p_{i-1}) &= -\rho u_{i-\frac{1}{2}}(u_i - u_{i-1}) - (p_i - p_{i-1}) \\ &= R_i^u \end{aligned} \quad (13)$$

We have also followed [3] in using a finite difference discretisation for the momentum equation.

For the continuity equation, we use a finite volume discretisation to give

$$\begin{aligned} A_{i+1}\rho u_{i+1} - A_i\rho u_i &= 0 \\ A_{i+1}\rho\delta u_{i+1} - A_i\rho\delta u_i &= -(A_{i+1}\rho u_{i+1} - A_i\rho u_i) \\ &= R_i^p \end{aligned} \quad (14)$$

Where R_u^i and R_p^i are the errors in satisfying the flow equations at node i , these are usually called the residuals.

We must also linearise the incompressible inlet boundary condition from Equation (7):

$$\begin{aligned} p_1 + \delta p_1 + \frac{1}{2}\rho(u_1 + \delta u_1)^2 &= P_0 \\ \delta p_1 + \rho u_1 \delta u_1 &= P_0 - p_1 - \frac{1}{2}\rho u_1^2 \\ \delta p_1 + \rho u_1 \delta u_1 &= R_1^u \end{aligned} \quad (15)$$

The exit boundary condition is simply:

$$\delta p_n = p_{exit} - p_n \quad (16)$$

Collecting together the discrete momentum and continuity equations leads to a set of simultaneous equations that can be expressed in block matrix form as:

$$\begin{pmatrix} A^u + B^{u_1} & D^x + B^{p_1} \\ M^x & B^{p_n} \end{pmatrix} \begin{pmatrix} |\delta u\rangle \\ |\delta p\rangle \end{pmatrix} = \begin{pmatrix} |R^u\rangle \\ |R^p\rangle \end{pmatrix} \quad (17)$$

Where

- A^u is the matrix containing the discrete convection operator for the u momentum equation.
- B^{u_1} and B^{p_1} are the the entries for the discretied inlet velocity boundary condition.
- D^x is the discrete pressure gradient operator.
- M^x is the the discrete continuity operator.
- B^{p_n} is the the entry for the discretised outlet pressure boundary condition.
- R^u and R^p are the residuals of the momentum and continuity equations.

Figure 4 shows the sparsity pattern for an implicit coupled incompressible matrix with 16 mesh nodes. Row 0 corresponds to the inlet boundary condition. Rows 1-15 are the momentum equations up to the exit node. Rows 16-30 are the continuity equations starting at the inlet node. Row 31 is the exit boundary condition.

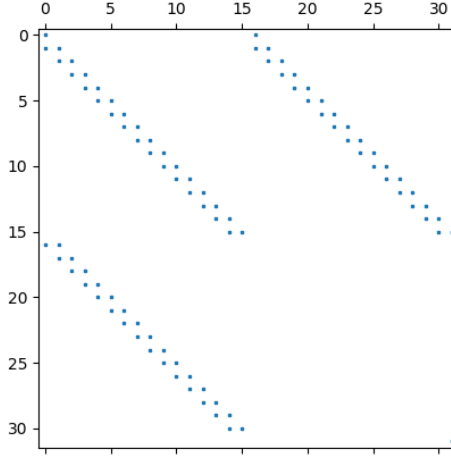


Figure 4: Incompressible nozzle with 16 stations, sparsity pattern for coupled matrix.

Table 1 provides details for a range of mesh sizes for incompressible implicit matrices. The dimensions of the Hermitian matrix are do the symmetrised matrix needed by HHL. The number of unitaries are derived from the decomposition of the Hermitian matrix into Pauli strings. The number of CFD iterations are those required to reduce the residuals and corrections to less than 1×10^{-12} .

Nstations	Hermitian matrix	λ_{min}	λ_{max}	κ	Nunitaries	CFD iterations
8	32x32	7.9×10^{-2}	3.32	42.1	128	5
16	64x64	3.9×10^{-2}	3.36	86.1	316	5
32	128x128	1.9×10^{-2}	3.38	173	740	5
64	256x256	9.6×10^{-3}	3.39	348	1,752	5

Table 1: Incompressible implicit matrices. Dimensions, eigenvalue range, condition number and number of unitaries in LCU. Sampled after at end of run. Area contraction = 10%.

2.2.2 Compressible coupled solver

For compressible flows, the implicit discretisation combines the gas law and energy equations to give:

$$\begin{aligned}
 p_i + \delta p_i &= R(\rho_i + \delta \rho_i)(T_0 - \frac{1}{2c_p}(u_i + \delta u_i)^2) \\
 &= R\rho_i(T_0 - \frac{1}{2c_p}(u_i^2 + 2u_i\delta u_i)) + R\delta\rho_i(T_0 - \frac{1}{2c_p}u_i^2) \\
 &= R\rho_i t_i + \frac{R}{c_p}\rho_i u_i \delta u_i + R t_i \delta \rho_i
 \end{aligned} \tag{18}$$

Collecting the terms together gives:

$$\begin{aligned} -\frac{R}{c_p}\rho_i u_i \delta u_i + \delta p_i - R t_i \delta \rho_i &= R \rho_i t_i - \delta p_i \\ &= R_\rho^i \end{aligned} \quad (19)$$

There is one further aspect to the implementation of the gas law which is due to the fact that the continuity control volumes are *downwinded* as shown in Figure 3b. To maintain stability at supersonic Mach numbers the perfect gas must compensate for this by using an upwinded effective pressure:

$$\rho = \frac{1}{R t} p_{eff} \quad (20)$$

For Mach numbers less than 2, this is discretised as [3]:

$$p_{eff,i} = p_{i-1} + a_0(p_i - p_{i-1}) + \frac{a_1}{2}(p_i - p_{i-2}) \quad (21)$$

Where

$$\begin{aligned} a_0 &= \frac{0.8}{3} \left(\frac{4}{M^2} - 1 \right) \\ a_1 &= 1.0 - a_0 \end{aligned} \quad (22)$$

Where M is the local maximum Mach number between stations i and $i - 1$. For low subsonic flow, $a_0 = 1$ and $a_1 = 0$ giving $p_{eff,i} = p_i$, i.e. no upwinding.

For the implicit discretisation, only the first order upwinding correction is retained. Hence, Equation (19) becomes:

$$-\frac{R}{c_p}\rho_i u_i \delta u_i + \delta p_i - R t_i \delta \rho_i = R \rho_i t_i - \delta p_{i-1} - a_0(\delta p_i - \delta p_{i-1}) \quad (23)$$

We must also linearise the compressible inlet boundary condition from Equation (5):

$$\begin{aligned} t_1 + \delta t_1 + \frac{1}{2c_p}(u_1 + \delta u_1)^2 &= T_0 \\ t_1 + \delta t_1 + \frac{1}{c_p}u_1 \delta u_1 &= T_0 - \frac{1}{2c_p}\rho u_1^2 \\ t_1 + \delta t_1 + \frac{1}{c_p}u_1 \delta u_1 &= R_1^u \end{aligned} \quad (24)$$

Linearising Equation (4) gives:

$$\begin{aligned} t_1 + \delta t_1 &= T_0 \left(\frac{p_1}{P_0} \right)^{\frac{\gamma-1}{\gamma}} \left(1 + \frac{\delta p_1}{p_1} \right)^{\frac{\gamma-1}{\gamma}} \\ &= T_0 \left(\frac{p_1}{P_0} \right)^{\frac{\gamma-1}{\gamma}} \left(1 + \frac{\gamma-1}{\gamma} \frac{\delta p_1}{p_1} \right) \end{aligned} \quad (25)$$

Combining Equation (24) and Equation (25) gives:

$$\begin{aligned} \frac{1}{c_p}u_1 \delta u_1 + \frac{T_0}{p_1} \left(\frac{p_1}{P_0} \right)^{\frac{\gamma-1}{\gamma}} \frac{\gamma-1}{\gamma} \delta p_1 &= T_0 - T_0 \left(\frac{p_1}{P_0} \right)^{\frac{\gamma-1}{\gamma}} - \frac{1}{2c_p}\rho u_1^2 \\ \frac{1}{c_p}u_1 \delta u_1 + \frac{T_0}{p_1} \left(\frac{p_1}{P_0} \right)^{\frac{\gamma-1}{\gamma}} \frac{\gamma-1}{\gamma} \delta p_1 &= R_1^u \end{aligned} \quad (26)$$

The exit boundary condition is the same as Equation (16).

The fixed point iteration means that the compressible discretisation is the same as the compressible one except that the local density is used rather than the constant one. The compressible continuity equation is straightforward application of corrections to the velocity and density. This leads to a coupled compressible matrix of the form:

$$\begin{pmatrix} A^u + B^{u_1} & D^x + B^{p_1} & 0 & 0 \\ M^u & B^{p_n} & M^p & 0 \\ L^u & L^p & L^\rho & 0 \\ 0 & 0 & 0 & I \end{pmatrix} \begin{pmatrix} |\delta u\rangle \\ |\delta p\rangle \\ |\delta \rho\rangle \\ |\psi\rangle \end{pmatrix} = \begin{pmatrix} |R^u\rangle \\ |R^p\rangle \\ |R^\rho\rangle \\ |0\rangle \end{pmatrix} \quad (27)$$

where the newly introduced terms are:

- L^u , L^p and L^ρ are the coefficients from the gas law discretisation.

- M^p are the additional terms from the discretisation of ρ in the continuity equation.
- I is a diagonal block that is required to extend the matrix so that it has a full rank equal to a power of 2. $|\psi\rangle$ are the corresponding entries in the state vector and the matrix solution ensures $|\psi\rangle = |0\rangle$. In practice the diagonal entries in I are set to have the same order as the other diagonal entries so as not to skew the eigenvalues.

Figure 5 shows the sparsity pattern for an implicit coupled compressible matrix with 16 mesh nodes.

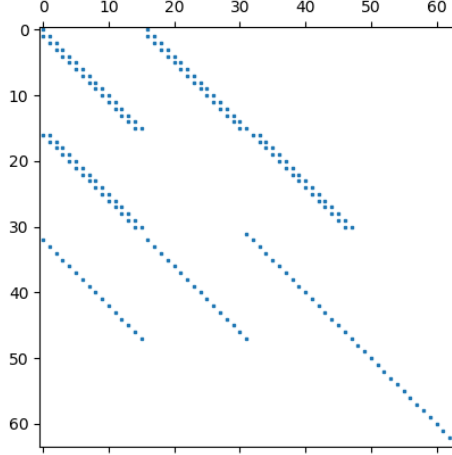


Figure 5: Compressible nozzle with 16 stations, sparsity pattern for coupled matrix.

Table 2 provides details for a range of mesh sizes for compressible implicit matrices. The dimensions of the Hermitian matrix are do the symmetrised matrix needed by HHL. The number of unitaries are derived from the decomposition of the Hermitian matrix into Pauli strings. The number of CFD iterations are those required to reduce the residuals and corrections to less than 1×10^{-12} .

Nstations	Hermitian matrix	λ_{min}	λ_{max}	κ	Nunitaries	CFD iterations
8	64x64	1.9×10^{-2}	3.01	157	416	96
16	128x128	8.9×10^{-3}	3.28	369	1,024	177
32	256x256	3.7×10^{-3}	3.51	941	2,432	410
64	512x512	1.4×10^{-3}	3.65	2,556	5,632	1,004

Table 2: Compressible implicit matrices. Dimensions, eigenvalue range, condition number and unitaries in LCU. Sampled after 100 iterations. Pressure ratio = 0.75

From the above derivation, it can be seen that there are additional highly non-linear terms in the compressible discretisation, particularly for the effective pressure. Whilst it is possible to recursively linearise these, it tends to produce an unstable discretisation. Indeed it is often necessary to apply relaxation to the initial coupled iterations and perform some pre-SIMPLE iterations to provide a better initial guess. The compressible implicit solver still provides a factor of 3-4 speed-up relative to the compressible SIMPLE solver, see Table 2 and Table 4, but is much less effective than for incompressible flows.

2.2.3 Incompressible SIMPLE solver

We follow the discretisation implemented in Appendix D of [3]. First of all we solve the momentum equation Equation (13) using a Jacobi scheme:

$$\delta u_i = \frac{R_u^i}{\rho u_{i-\frac{1}{2}}} \quad (28)$$

All the δu_i terms are evaluated at each station before being used to update the velocities. This equation is solved on the classical device as the calculation of δu_i can be overlapped with the matrix assembly. Note that the pressures are not updated at this stage as they are done separately using the pressure correction equation. The SIMPLE algorithm falls into a broader class of *segregated* solves.

To solve the continuity equation, the momentum equation is assumed to have been solved, allowing the velocity and pressure corrections to be related by:

$$\begin{aligned}\delta u_i &= -\frac{1}{\rho u_{i-\frac{1}{2}}}(\delta p_i - \delta p_{i-1}) \\ \delta u_{i+1} &= -\frac{1}{\rho u_{i+\frac{1}{2}}}(\delta p_{i+1} - \delta p_i)\end{aligned}\quad (29)$$

Note that the first equation is based on the momentum equation for station i and the second equation on the one at for station $i + 1$. These equations can be substituted into the continuity equation Equation (14) at station i to give

$$-\frac{A_{i+1}}{u_{i+\frac{1}{2}}}(\delta p_{i+1} - \delta p_i) + \frac{A_i}{u_{i-\frac{1}{2}}}(\delta p_i - \delta p_{i-1}) = R_p^i \quad (30)$$

Setting $a_i = \frac{A_i}{u_{i-\frac{1}{2}}}$, Equation (30) can be written as:

$$-a_i \delta p_{i-1} + (a_i + a_{i+1}) \delta p_i - a_{i+1} \delta p_{i+1} = R_p^i \quad (31)$$

This leads to a matrix equation of the form $A^p |\delta p\rangle = |R^p\rangle$ which has the sparsity pattern shown Figure 6.

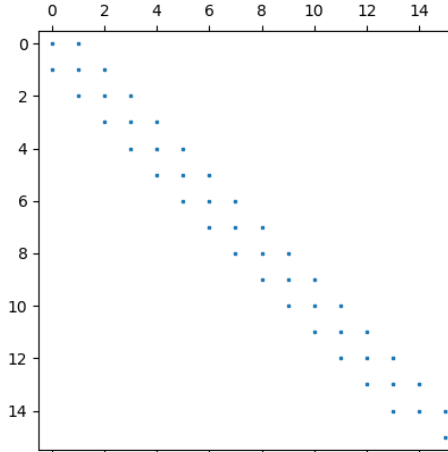


Figure 6: Incompressible nozzle with 16 stations, sparsity pattern for pressure correction matrix.

Once the pressure correction has been solved, $|\delta p\rangle$ is used to an update to the velocity field using corrections according to Equation (29). Since the continuity equation is linear for incompressible flows, this results in a velocity field that exact satisfies continuity. This step is also completed on the classical device as it does not require a matrix solution.

Table 3 provides details for a range of mesh sizes for incompressible pressure corrections matrices. The dimensions of the Hermitian matrix are do the symmetrised matrix needed by HHL. The number of unitaries are derived from the decomposition of the Hermitian matrix into Pauli strings. The number of CFD iterations are those required to reduce the residuals and corrections to less than 1×10^{-12} .

2.2.4 Compressible SIMPLE solver

The compressible pressure correction equation follows the derivation of the incompressible equation except that the additional terms in $\delta \rho$ are replaced by δp terms using the the gas law. We follow [3] and used a fully upwinded effective pressure for the gas law. This produces a pressure correction matrix with the same sparsity pattern as Figure 6.

Nstations	Hermitian matrix	λ_{min}	λ_{max}	κ	Nunitaries	CFD iterations
8	16x16	1.4×10^{-1}	3.18	23.3	24	194
16	32x32	3.5×10^{-2}	3.36	97.4	56	389
32	64x64	8.7×10^{-3}	3.56	410	128	765
64	128x128	2.1×10^{-3}	3.81	1,747	288	1,491

Table 3: Incompressible pressure correction matrices. dimensions, eigenvalue range, condition number and unitaries in LCU. Sampled after after 10 iterations. Area contraction = 10%.

Table 4 provides details for a range of mesh sizes for compressible pressure corrections matrices. The dimensions of the Hermitian matrix are do the symmetrised matrix needed by HHL. The number of unitaries are derived from the decomposition of the Hermitian matrix into Pauli strings. The number of CFD iterations are those required to reduce the residuals and corrections to less than 1×10^{-12} .

Nstations	Hermitian matrix	λ_{min}	λ_{max}	κ	Nunitaries	CFD iterations
8	16x16	4.8×10^{-1}	6.89	14.3	32	424
16	32x32	1.9×10^{-1}	8.22	42.9	80	700
32	64x64	7.1×10^{-2}	9.25	131	192	1,449
64	128x128	3.0×10^{-2}	10.1	341	448	2,884

Table 4: Compressible pressure correction matrices. Dimensions, eigenvalue range, condition number and unitaries in LCU for the compressible nozzle.(sampled after 100 iterations).

3 Core CFD solver

The core CFD solver is written in C with a Python wrapper written using ctypes. The C code is self contained and does not rely on any third party libraries other than those provided by the compiler. e.g. libm on Linux.

3.1 Building the C code

3.1.1 Linux based systems

The C code can be built as a stand-alone executable or as a library to be linked to a Python code.

- To make the C executable, type `make` in the `src` directory. The executable `nozzle` is placed in the `bin` directory.
- To make the C library, type `make lib` in the `src` directory. The shared object library `libnozzle.so` is placed in the `lib` directory.
- To rebuild from scratch , type `make clean` in the `src` directory.

3.1.2 Windows based systems

Not cuurently supported.

3.2 Running the code

There are 3 modes of running the code include in the distribution.

- To run the C-code directly type: `[path]/bin/nozzle -i [path]inputfile.xml`.

Typing `[path]/bin/nozzle -h` gives a short list of the input options.

Sample input files are included in the `input` directory of the distribution. Note each input file can be run in the four possible combinations of incompressible/compressible flow and SIMPLE/coupled solvers. Which combination is run is determined by the `setup` section of the input file as described in Section 3.5.

- To run the C solver as a library call from Python type: `[path]/script/run_cprog.py -i [path]inputfile.xml`. This simply demonstrates the infrastructure used to call the C library from within a Python script.
- To use the C library to set-up matrices to be solved using the `scipy` linear algebra routine type: `[path]/script/run_pysolve.py -i [path]inputfile.xml`.

The final option provides a template for replacing the `scipy` linear solver with a quantum linear equation solver as described in Section 3.4.

3.3 Python-C interface

There are two layers to the Python-C interface:

- Data instantiation when python is the main routine.
- Data exchange to *get* a matrix and a right-hand side vector, and to *put* a solution vector.

Both of these depend on sharing C structures with the Python code.

3.3.1 Data instantiation

Listing 1 shows the high level structure used by the C solver. As can be seen, this is a structure of structures which, collectively, contain all the data used by the solver. These are not further defined here as they are not needed by the Python layer.

```
1 typedef struct
2 {
3     SETUP    *setup;
4     MESH     *mesh;
5     FLOW     *flow;
6     SOLVER   *solver;
7     FILES    *files;
8     ARRAYS   *arrays;
9     HISTORY  *history;
10 } NOZZLE;
```

Listing 1: C master structure for nozzle data

In fact, the Python layer needs only manage the pointer which it can do by treating it as a `void`. The only data passed between the C and Python layers is the memory address of the nozzle pointer. Listing 2 gives code segments which call C routines to instantiate an empty nozzle structure; load data from an input file; and, free the structure after the calculations have been completed. Note that the call to `c_lib.c2py_noz_create` on line 2 passes the pointer by reference. This is needed because the memory for the pointer is allocated with the C code, so it must return a pointer to a pointer. The variable `i_cptr` on line 4 is the name of the input file that is read as a command line option.

```
1 noz_cptr = c_void_p()
2 c_lib.c2py_noz_create(byref(noz_cptr))
3 ...
4 c_lib.c2py_read_input(i_cptr, noz_cptr)
5 ...
6 c_lib.c2py_noz_free(noz_cptr)
```

Listing 2: Python instantiation.

3.3.2 Data exchange

In order to investigate quantum linear solvers, it is necessary to share matrices and vectors between the C and Python layers. This is done by using a common structure definition. Listing 3 shows the C structure and Listing 4 shows the corresponding Python structure. Both used the same compressed row format.

```

1 typedef struct
2 {
3     int32_t nr;
4     int32_t nc;
5     int32_t nnz;
6     int32_t *col;
7     int32_t *rst;
8     double *val;
9     double *b;
10 } SMATRIX;

```

Listing 3: C sparse matrix structure.

Note that integers are declared as `int32` to ensure both the C and Python layers use 32 bit integers. Note that the sparse matrix structures also includes the residual, or right hand side vector, `b`. This is a convenience as both the matrix and vector can be retrieved in a single call. It is also convenient on the C side as the matrix entries and residuals are computed at the same time. The solution vector is computed separately and, therefore, not included in the structure.

```

1 class SMATRIX(Structure):
2     _fields_ = [("nr", c_int32),
3                 ("nc", c_int32),
4                 ("nnz", c_int32),
5                 ("col", POINTER(c_int32)),
6                 ("rst", POINTER(c_int32)),
7                 ("val", POINTER(c_double)),
8                 ("b", POINTER(c_double))
9                 ]
10     _defaults_ = {"nr" : 0,
11                  "nc" : 0,
12                  "nnz": 0,
13                  "col": None,
14                  "rst": None,
15                  "val": None,
16                  "b"  : None}

```

Listing 4: Python sparse matrix structure.

Listing 5 shows how to *get* a matrix and vector from the C code and convert them to a scipy sparse matrix and numpy vector. This is taken from the script `run_pysolve.py`. On line 10, the `scipy.sparse.linalg.spsolve` routine is used to solve for `x`. Lines 12-13 show how to *put* the solution vector back into the C code.

```

1 S = SMATRIX()
2 c_lib.c2py_get_smat(noz_cptra, byref(S))
3
4 col = np.ctypeslib.as_array(S.col, shape=(S.nnz,))
5 rst = np.ctypeslib.as_array(S.rst, shape=(S.nr+1,))
6 val = np.ctypeslib.as_array(S.val, shape=(S.nnz,))
7 b   = np.ctypeslib.as_array(S.b, shape=(S.nr,))
8
9 SP = sparse.csr_matrix((val, col, rst), shape=(S.nr, S.nc), dtype=np.double)
10 x  = spsolve(SP, b)
11
12 xc = x.ctypes.data_as(POINTER(c_double))
13 rc = c_lib.c2py_put_soln(noz_cptra, xc)

```

Listing 5: Code segment for Python-C matrix and vector exchange.

As can be seen, the above code segments are independent of the CFD flow type and solution scheme. These are contained within the C layer and controlled via a single input file.

3.4 Linking to a quantum linear equation solver

In principle, linking to a quantum solver is *just* a matter of replacing the call to `spsolve` in line 10 of Listing 5 with a call to the equivalent quantum solver. However, the quantum solver is likely to need a lot of additional input, including additional data such as phase factors for QSVT. The user has a choice to hardwire them for initial development testing, add them as additional input options to the python script, or include them in an auxiliary input file.

Note that the call to `spsolve` is within an iterative loop. Only data that changes as part of the non-linear outer iterations should be included within the loop. Data input and set-up should be outside the loop.

It is recommended to make a copy of the file `run_pysolve.py` and use that as a template for the QLES solver.

3.5 Input file

There are sample input files in the `input` directory of the distribution. The files are structured to include options for all combinations of flow type and solver. The choice to be run is controlled by the `setup` section as described in Section 3.5.1. If files need to be simplified, the sections for the unused options can be removed.

3.5.1 Overview

The input file has been designed for all 4 combinations of flow regime and solver type to be held in a single file with two binary flags to select which option is executed. This adds some verbosity to the file but eases the number of files that need to be managed. This does not prevent separate files being used for separate cases if that is preferred. The high-level structure of the file is shown in Listing 6.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <nozzle>
3   <setup>
4     <casename>nozzle</casename>
5     <compressible>0</compressible>
6     <couple>1</couple>
7   </setup>
8
9   <mesh>...</mesh>
10
11   <incompressible>
12     <flow>...</flow>
13     <solver>
14       <simple>...</simple>
15       <couple>...</couple>
16     </solver>
17   </incompressible>
18
19   <compressible>
20     <flow>...</flow>
21     <solver>
22       <simple>...</simple>
23       <couple>...</couple>
24     </solver>
25   </compressible>
26 </nozzle>
```

Listing 6: Input file main sections.

The `setup` section contains a case name and binary variables to toggle the flow and solver types. This section in Listing 6 specifies incompressible flow using the coupled solver. See Section 3.6.1 for how the case name is used to set the names of the output files based on the setup.

The `mesh` section is common to all setup combinations and is described below. There are separate sections for incompressible and compressible flow with each containing `flow` and `solver` sections. The solver sections contain separate `simple` and `couple` sections for each solver type. As shown below, the

variables in each section depend on the set-up and where variables have the same name they generally have different values, according to the section they are in.

All variables have default values. Those variables that use the default values can be omitted from the input file for brevity, if desired.

3.5.2 Mesh data

Listing 7 shows the `mesh` section of the input file.

```

1 <mesh>
2   <nx>16</nx>
3   <ain>1.0</ain>
4   <da>-0.1</da>
5 </mesh>
```

Listing 7: Input file mesh section.

The variables are:

- **nx** - the number of axial stations in the mesh, this value is used as part of the output file names.
- **ain** - for incompressible flows this is the inlet area of the channel. For compressible flows this is a scale factor for the inlet area, which can be used to scale the matrix entries but is usually set to 1.0.
- **da** - the area contraction at the midpoint of the channel for incompressible flows. This is specified as a fraction of the inlet area and is negative for an area contraction and positive for an area expansion.

3.5.3 Incompressible flow, SIMPLE solver

Listing 8 shows the section of the input file for incompressible flow with the SIMPLE solver. Note that all flow variables are non-dimensional.

```

1 <incompressible>
2   <flow>
3     <rho>1.0</rho>           <!-- constant density -->
4     <bc>
5       <ptin>1.5</ptin>       <!-- inlet total pressure -->
6       <pex>0.95</pex>        <!-- exit static pressure -->
7     </bc>
8   </flow>
9
10  <solver>
11    <simple>
12      <niters>2000</niters>    <!-- number of iterations -->
13      <relaxu>1.0</relaxu>    <!-- u-relaxation factor -->
14      <relaxp>1.0</relaxp>    <!-- p-relaxation factor -->
15      <toleru>1.0e-12</toleru> <!-- u-convergence tolerance -->
16      <tolerp>1.0e-12</tolerp> <!-- p-convergence tolerance -->
17    </simple>
18  </solver>
19 </incompressible>
```

Listing 8: Input file for incompressible + SIMPLE solver.

The variables are defined in Listing 8 and should be self-explanatory. As shown in Table 3, the number of iterations varies considerably for different mesh sizes. **niters** is the maximum number of iterations, the calculation will terminate when it reaches either **niters** or both of the convergence tolerances have been met.

3.5.4 Incompressible flow, coupled solver

Listing 9 shows the section of the input file for incompressible flow with the coupled solver. Note that all flow variables are non-dimensional.

```

1 <incompressible>
2   <flow>... </flow>
3
4   <solver>
5     <couple>
6       <niters>10</niters>           <!-- number of iterations -->
7       <presim>0</presim>           <!-- number SIMPLE iterations before coupled solver -->
8       <relaxc>1.0</relaxc>         <!-- relaxation factor applied to all variables -->
9       <rrampc>0</rrampc>           <!-- number of iterations over which to ramp relaxc -->
10      <tolerc>1.0e-12</tolerc>      <!-- convergence tolerance for all variables -->
11    </couple>
12  </solver>
13 </incompressible>

```

Listing 9: Input file for incompressible + coupled solver.

The `flow` is not included as it is identical to Listing 8. The number of iterations is as before, but has a much smaller value as Table 2 shows that number of iterations is less than 10 for all mesh sizes. A single convergence tolerance, `tolerc`, is used for the coupled set of equations. The variables `presim`, `relaxc` and `rrampc` are needed for the compressible coupled solver and are described in Section 3.5.6. The values of these variables in Listing 9 are the defaults and these can be omitted from the input file for incompressible flows.

3.5.5 Compressible flow, SIMPLE solver

Listing 10 shows the section of the input file for compressible flow with the SIMPLE solver. Note that all flow variables are non-dimensional.

```

1 <compressible>
2   <flow>
3     <bc>
4       <ptin>1.5</ptin>             <!-- inlet total pressure -->
5       <min>0.8</min>               <!-- inlet Mach number -->
6       <pex>0.75</pex>              <!-- exit static pressure -->
7     </bc>
8     <gaslaw>
9       <gamma>1.4</gamma>            <!-- ratio of specific heats -->
10      <cp>3.5</cp>                  <!-- specific heat at constant pressure -->
11      <rgas>1.0</rgas>              <!-- perfect gas constant -->
12      <upwind>2</upwind>            <!-- 1, 2, 3 for 1st, 2nd and 3-pt density upwinding -->
13    </gaslaw>
14  </flow>
15
16  <solver>
17    <simple>
18      <niters>5000</niters>          <!-- number of iterations -->
19      <relaxu>1.0</relaxu>          <!-- u-relaxation factor -->
20      <relaxp>1.0</relaxp>          <!-- p-relaxation factor -->
21      <toleru>1.0e-12</toleru>       <!-- u-convergence tolerance -->
22      <tolerp>1.0e-12</tolerp>       <!-- p-convergence tolerance -->
23    </simple>
24  </solver>
25 </compressible>

```

Listing 10: Input file for incompressible + SIMPLE solver.

3.5.6 Compressible flow, coupled solver

Listing 11 shows the section of the input file for compressible flow with the coupled solver. Note that all flow variables are non-dimensional.

```

1 <compressible>

```



```

2    <flow>...</flow>
3
4    <solver>
5      <couple>
6        <niters>2000</niters>      <!-- number of iterations -->
7        <presim>10</presim>      <!-- number SIMPLE iterations before coupled solver -->
8        <relaxc>0.75</relaxc>    <!-- relaxation factor applied to all variables -->
9        <rrampc>20</rrampc>      <!-- number of iterations over which to ramp relaxc -->
10       <tolerc>1.0e-12</tolerc> <!-- convergence tolerance for all variables -->
11     </couple>
12   </solver>
13 </compressible>

```

Listing 11: Input file for incompressible + coupled solver.

3.6 Output files

3.6.1 File naming convention

There are two output files: a convergence history and a flow solution file. The naming convention for both files is *casename.nx_ffss.ext*, where:

- *casename* is the user specified name in the input file.
- *nx* is the number of stations.
- *ff* is a two character identifier for the flow type: *in* for incompressible flow or *co* for compressible flow.
- *ss* is a two character identifier for the solver type: *si* for the SIMPLE solver or *co* for the coupled solver.
- *ext* is the three character file extension: *hst* for the convergence history file and *sol* for the solution file.

For example, the history file for an incompressible SIMPLE case named 'nozzle' with 16 stations is *nozzle_016_insi.hst*. If cases with more than 999 stations are needed, modify the routine *noz_files* to change the format statement in the *snprintf* statements to increase '3' to the number of digits needed.

3.6.2 Convergence history

The contents of the convergence history file depend on the flow type. In both cases the file contains the root mean square (RMS) of the changes in the flow variables between the current iteration and the previous one; and, the residual errors, e.g. R^u , of the flow equations at the start of the iteration. Both should reduce towards the convergence tolerance as the iterations proceed. For incompressible cases, the history file contains the RMS changes for u and p and the RMS residuals of the momentum and continuity equations. For compressible flows, the RMS changes in density and residual of the gas equation are also included. Sample files for both flow types are listed below, showing the first 9 iterations.

See Section 4.1 for details on how to plot the convergence history of a single run and to compare the convergence histories of multiple runs.

```

head nozzle_016_insi.hst
"iter",  "du_rms",      "dp_rms",      "resu_rms",      "resp_rms"
1,  0.00000000e+00,  9.95624279e-02,  0.00000000e+00,  1.62406810e-02
2,  3.42680753e-02,  9.21423809e-02,  3.56263102e-02,  2.32527299e-02
3,  4.09420463e-03,  1.56536726e-03,  4.31829249e-03,  1.56084554e-03
4,  3.82501330e-03,  2.49477798e-03,  4.06783477e-03,  8.77342756e-04
5,  3.60318337e-03,  2.36292698e-03,  3.84504091e-03,  8.24039930e-04
6,  3.39492043e-03,  2.23327519e-03,  3.63437959e-03,  7.76310997e-04
7,  3.19933160e-03,  2.11074512e-03,  3.43527772e-03,  7.31505443e-04
8,  3.01557136e-03,  1.99495898e-03,  3.24709921e-03,  6.89419128e-04
9,  2.84285931e-03,  1.88554307e-03,  3.06924290e-03,  6.49871476e-04

```

```
head nozzle_016_coco.hst
```

"iter",	"du_rms",	"dp_rms",	"dr_rms",	"resu_rms",	"resp_rms",	"resr_rms"
1,	2.29801e-01,	5.03378e-02,	2.88455e-02,	1.07241e-02,	2.03448e-02,	2.03448e-02
2,	2.58647e-01,	1.51207e-01,	1.17747e-01,	5.94100e-03,	5.40709e-03,	5.40709e-03
3,	8.72627e-01,	5.95568e-01,	4.75178e-01,	4.18458e-03,	6.25604e-03,	6.25604e-03
4,	3.39289e-01,	2.75104e-01,	2.11024e-01,	1.35157e-01,	2.57616e-01,	2.57616e-01
5,	1.13501e+00,	5.81302e-01,	3.79980e-01,	1.81180e-01,	5.31819e-02,	5.31819e-02
6,	4.67940e-01,	2.83047e-01,	2.98249e-01,	1.93246e-01,	3.24966e-01,	3.24966e-01
7,	2.68325e-01,	1.12835e-01,	9.90863e-02,	5.66960e-02,	1.91540e-01,	1.91540e-01
8,	2.00619e-01,	1.24138e-01,	7.90677e-02,	2.57842e-02,	6.30679e-02,	6.30679e-02
9,	3.47459e-01,	2.28170e-01,	1.64274e-01,	8.91253e-03,	1.89085e-02,	1.89085e-02

Note that the precision of the compressible file has been truncated for the purposes of illustrating the file format. The actual file has the same precision as the incompressible file.

3.6.3 Flow field

As with the convergence histories, the format of the solution files depends on whether incompressible or compressible flow has been modelled. Sample files for both flow types are listed below, showing the solution with 8 axial stations.

See Section 4.2 for details on how to plot the flow field of a single run and to compare the flow fields of multiple runs.

```
head nozzle_008_insi.sol
```

"index",	"x",	"area",	"density",	"velocity",	"pressure"
1,	0.00000000,	1.00000000,	1.00000000,	0.99352315,	1.00645586
2,	0.14285714,	0.95002040,	1.00000000,	1.05396039,	0.93273107
3,	0.28571428,	0.91778253,	1.00000000,	1.09468920,	0.87557899
4,	0.42857142,	0.90196380,	1.00000000,	1.12077187,	0.83702077
5,	0.57142857,	0.90196380,	1.00000000,	1.12039273,	0.83109312
6,	0.71428571,	0.91778253,	1.00000000,	1.10510955,	0.84489365
7,	0.85714285,	0.95002040,	1.00000000,	1.06576098,	0.88937807
8,	1.00000000,	1.00000000,	1.00000000,	1.01185327,	0.95000000

```
head nozzle_008_cosi.sol
```

"index",	"x",	"area",	"density",	"velocity",	"pressure",	"temp",	"Mach no"
1,	0.000000,	1.038230,	0.733619,	0.903193,	0.648126,	0.883463,	0.812124
2,	0.142857,	1.002814,	0.673123,	1.019130,	0.569949,	0.851624,	0.933340
3,	0.285714,	1.005860,	0.594401,	1.150608,	0.479892,	0.810871,	1.079907
4,	0.428571,	1.039360,	0.524993,	1.260737,	0.405785,	0.772934,	1.211963
5,	0.571428,	1.099747,	0.448136,	1.395860,	0.318806,	0.721653,	1.388716
6,	0.714285,	1.185954,	0.441964,	1.312472,	0.369072,	0.753916,	1.277511
7,	0.857142,	1.298492,	0.629288,	0.841891,	0.630211,	0.898745,	0.750540
8,	1.000000,	1.438982,	0.791263,	0.604183,	0.750000,	0.947851,	0.524487

As before, the precision of the output has been truncated for the purposes of illustrating the file format.

4 Analysing the results

4.1 Convergence histories

Up to 5 convergence history files can be compared using the script `tt plot_hist.py`. As described in Section 3.6.2 the format of the plot depends on the type of flow.

Figure 7 illustrates how to compare the convergence of the coupled compressible solver on three different meshes using the following command:

```
./plot_hist.py nozzle_008_coco.hst nozzle_016_coco.hst nozzle_032_coco.hst
```

The file names are used as the legends for each line in the plot.

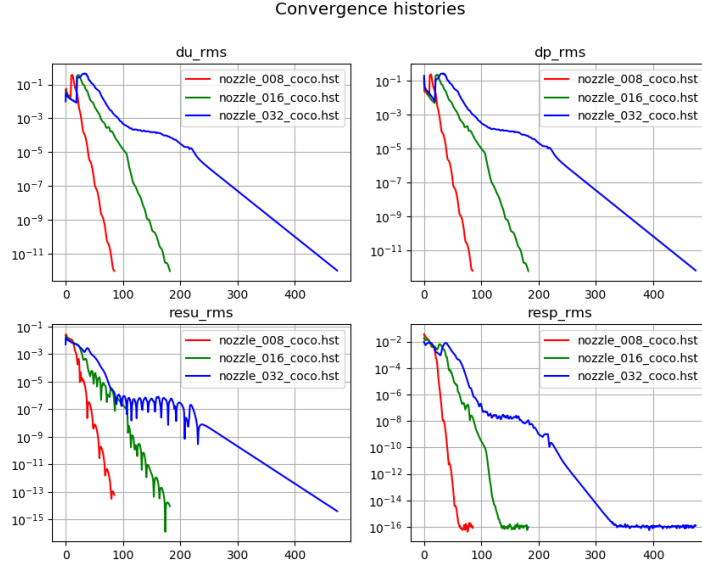


Figure 7: Convergence history plot for the coupled compressible solver on three different meshes.

4.2 Flow field

Up to 5 flow solution files can be compared using the script `tt plot_sol.py`. As described in Section 3.6.3 the format of the plot depends on the type of flow.

Figure 8 illustrates how to compare the solutions of the coupled compressible solver on three different meshes using the following command:

```
./plot_sol.py nozzle_008_coco.sol nozzle_016_coco.sol nozzle_032_coco.sol
```

The plots illustrate how the shock wave becomes sharper as the resolution of the mesh increases.

Figure 9 illustrates how to compare the solutions of the coupled incompressible solver on three different meshes using the following command:

```
./plot_sol.py nozzle_008_inco.sol nozzle_016_inco.sol nozzle_032_inco.sol
```

This illustrates different nature of the incompressible where the flow is symmetric about the centre of the nozzle.

5 Summary

The 1D nozzle solver provides a flexible Python interface for testing quantum linear equation solvers on fluid flow type problems. The core CFD solver, written in C, is linked via a library to enable developers to focus on their quantum solver. The benefit of the nozzle system is that it allows researchers to investigate the feedback between the inner linear system, solved using a quantum algorithm, and the outer classical non-linear update.

Although the solver is only 1 dimensional, there are a range of solver options that can be used to generate matrices with up to 8 diagonals using the coupled compressible solver. The standard pressure correction solver has 3 diagonals. The coupled incompressible solver has 6 diagonals and gives rapid convergence which is useful when emulating the quantum circuit is time-consuming.

The plotting routines enable easy comparison of classical and quantum results.

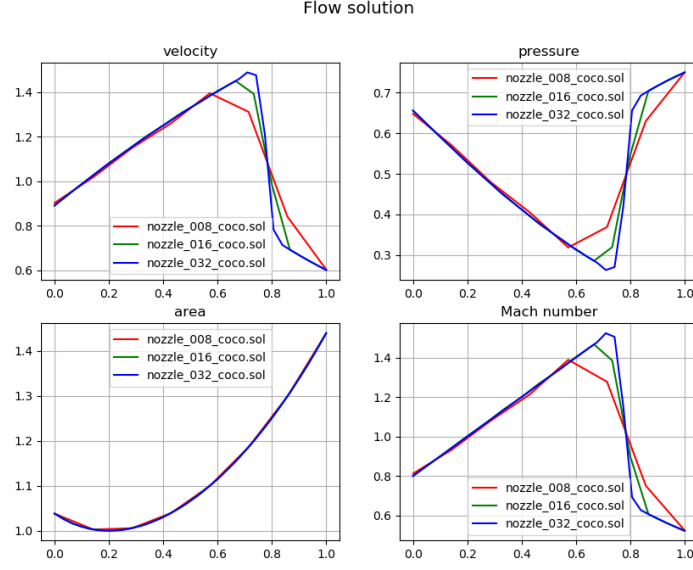


Figure 8: Flow solution plot for the coupled compressible solver on three different meshes.

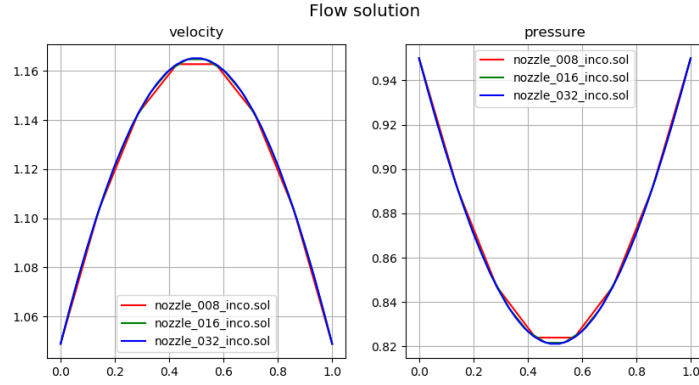


Figure 9: Flow solution plot for the coupled incompressible solver on three different meshes.

References

- [1] Yulong Dong, Xiang Meng, K Birgitta Whaley, and Lin Lin. Efficient phase-factor evaluation in quantum signal processing. *Physical Review A*, 103(4):042419, 2021.
- [2] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 193–204, 2019.
- [3] Ajay K Gupta. *Development of a new shock capturing formula for pressure correction methods*. PhD thesis, Virginia Tech, 1993.
- [4] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical review letters*, 103(15):150502, 2009.
- [5] Leigh Lapworth. Implicit hybrid quantum-classical cfd calculations using the hhl algorithm. *arXiv preprint arXiv:2209.07964*, 2022.
- [6] John M Martyn, Zane M Rossi, Andrew K Tan, and Isaac L Chuang. Grand unification of quantum algorithms. *PRX Quantum*, 2(4):040203, Dec 2021.